ECG CSV BLE 転送 仕様書

2026-10-25 · Hideo

iPad で選んだ心電図 CSV を Bluetooth Low Energy（BLE）で iPhone に送り、iPhone で波形と異常検知を表示する Pythonista 3 アプリの仕様です。対象版は **2026-10-25 r17** で、実機（iPad → iPhone）で転送と表示を確認済みです。

1. 概要

心電データ（256 Hz）の CSV を、Wi-Fi やネットワーク設定なしで iPad から iPhone へ送り、その場で波形と異常を確認するためのアプリです。iPhone はサービス UUID で iPad を自動で見つけて接続します（IP アドレス・端末名の設定は不要）。

項目	内容
送信側	iPad（`ecg_sender_ipad.py`）／BLE ペリフェラル
受信・表示側	iPhone（`ecg_viewer_iphone.py`）／BLE セントラル
実行環境	Pythonista 3.4（Python 3.10.4）、iOS / iPadOS の Bluetooth をオン
外部ライブラリ	なし（Pythonista 標準の `ui`・`dialogs`・`console`・`objc_util` と CoreBluetooth）
対象ファイル	心電 CSV（例: `ECGtsHDdata1.csv`）、1 ファイル最大 20 MB
通信	BLE の Notify（iPad → iPhone）と Write（iPhone → iPad の受信結果）
版	2026-10-25 r17（起動時に画面へ表示）
保管場所	GitHub `HIdeo-Okano/ai-camera-app` の `pythonista/ecg_ble/`、Google Drive「Pytest」フォルダー

2. 構成

端末に置くのは 2 ファイルだけで、どちらも 1 ファイルで動きます。共通処理（解析・CSV・BLE）は両方に同じものを埋め込んでいます。

ファイル	置き場所	役割
`ecg_sender_ipad.py`	iPad：このiPad内 > Pythonista 3	CSV を選び、4行ヘッダにそろえて BLE で送る
`ecg_viewer_iphone.py`	iPhone：このiPhone内 > Pythonista 3	BLE で受信・保存し、波形と異常検知を表示する
`ecg_ble_section.py`	開発用	BLE の共通部分の原本（上の 2 ファイルに同じものを埋め込む）
`test_ecg_ble.py`	開発用（PC）	CoreBluetooth を模擬した自動テスト（18 件）
`ble_diag_ipad.py`	開発用（iPad）	BLE の準備を 1 段ずつ行い、iPhone の接続を待つ診断
`ble_diag_sender_start.py`	開発用（iPad）	送信プログラムの起動診断
`README.md`	—	使い方の説明書

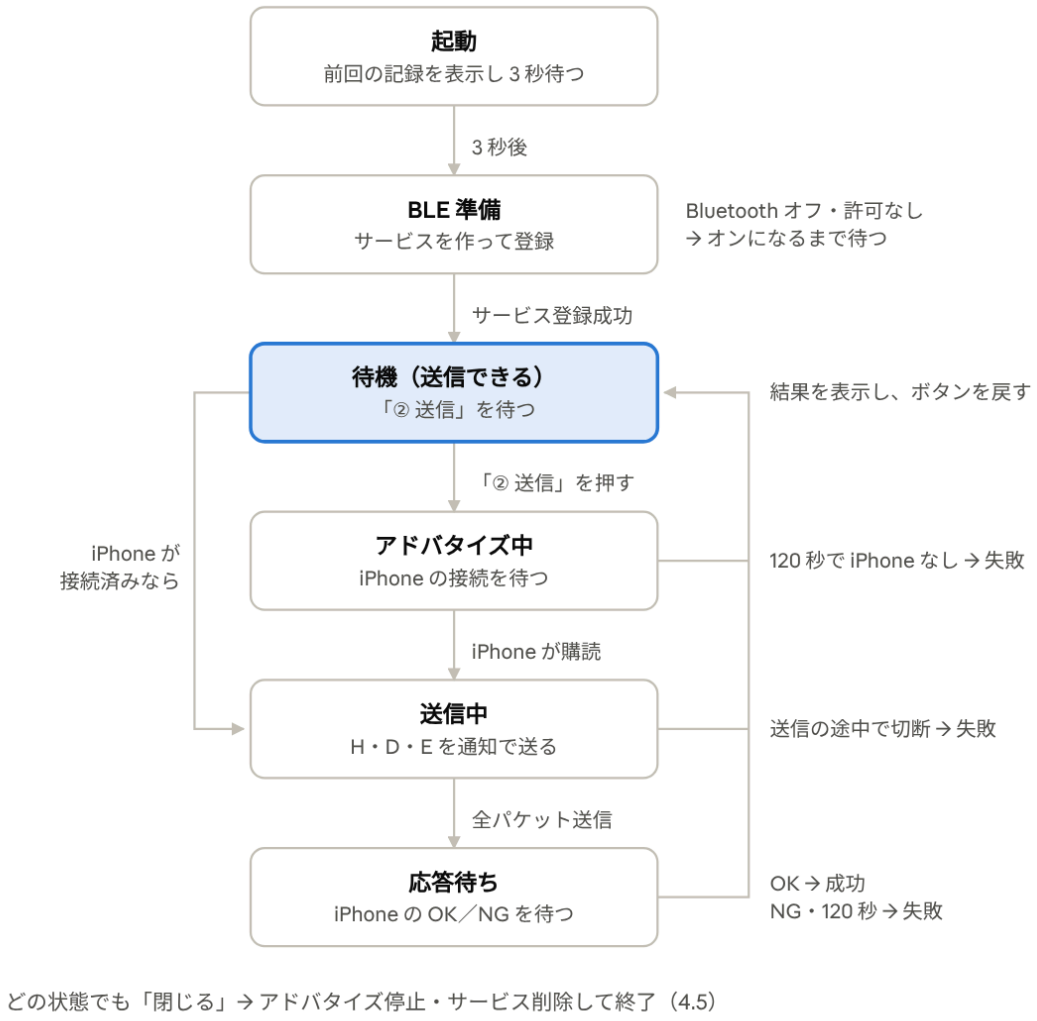
スクリプトは必ず Pythonista 自身のフォルダーに置き、Pythonista のファイル一覧から実行します。「ファイル」アプリから直接開くと、同じフォルダーへの書き込みができません（PermissionError）。

3. システム概要（状態遷移図）

iPad は「待機」、iPhone は「探索中」を中心に動き、1 回の送受信が終わるとどちらも自動でそこへ戻ります。失敗したときも、画面に理由を出してから戻ります。

3.1 iPad（送信）の状態遷移

iPad は送信が成功しても失敗しても待機に戻る

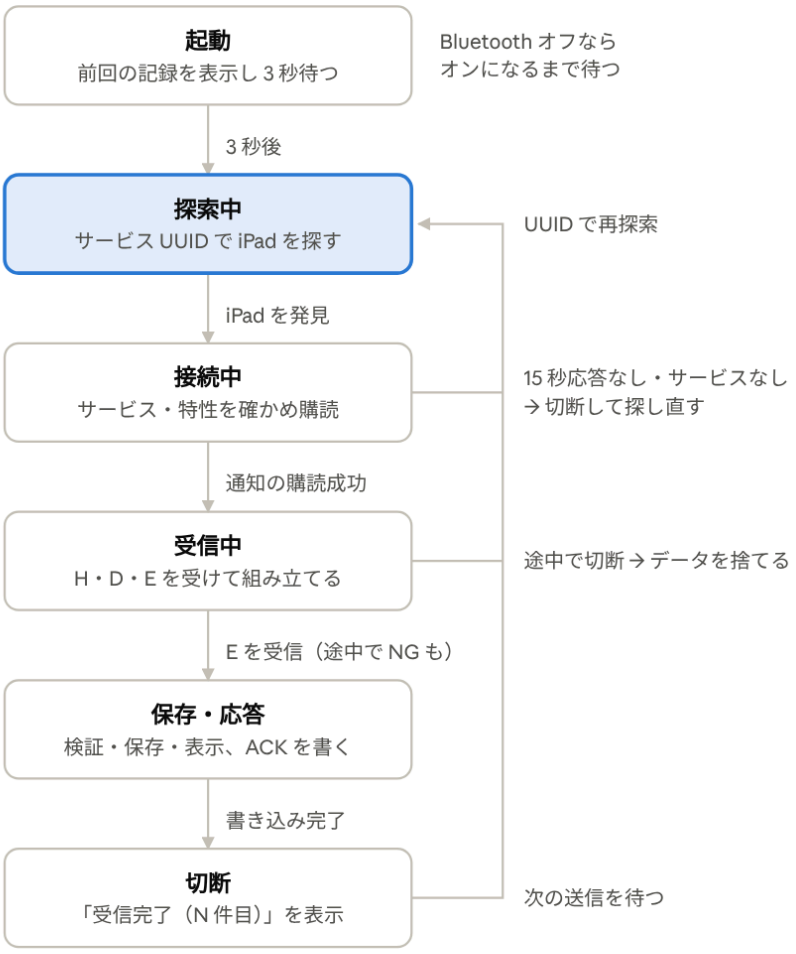


iPad（送信）の状態遷移 · 6 状態

「② 送信」を押したとき iPhone がすでに接続していれば、アドバタイズを飛ばして送信中に進みます。

3.2 iPhone（受信）の状態遷移

iPhone は 1 回受信するごとに切断して探索中に戻る



どの状態でも「×」→ 探索・接続を止めて終了（4.5）

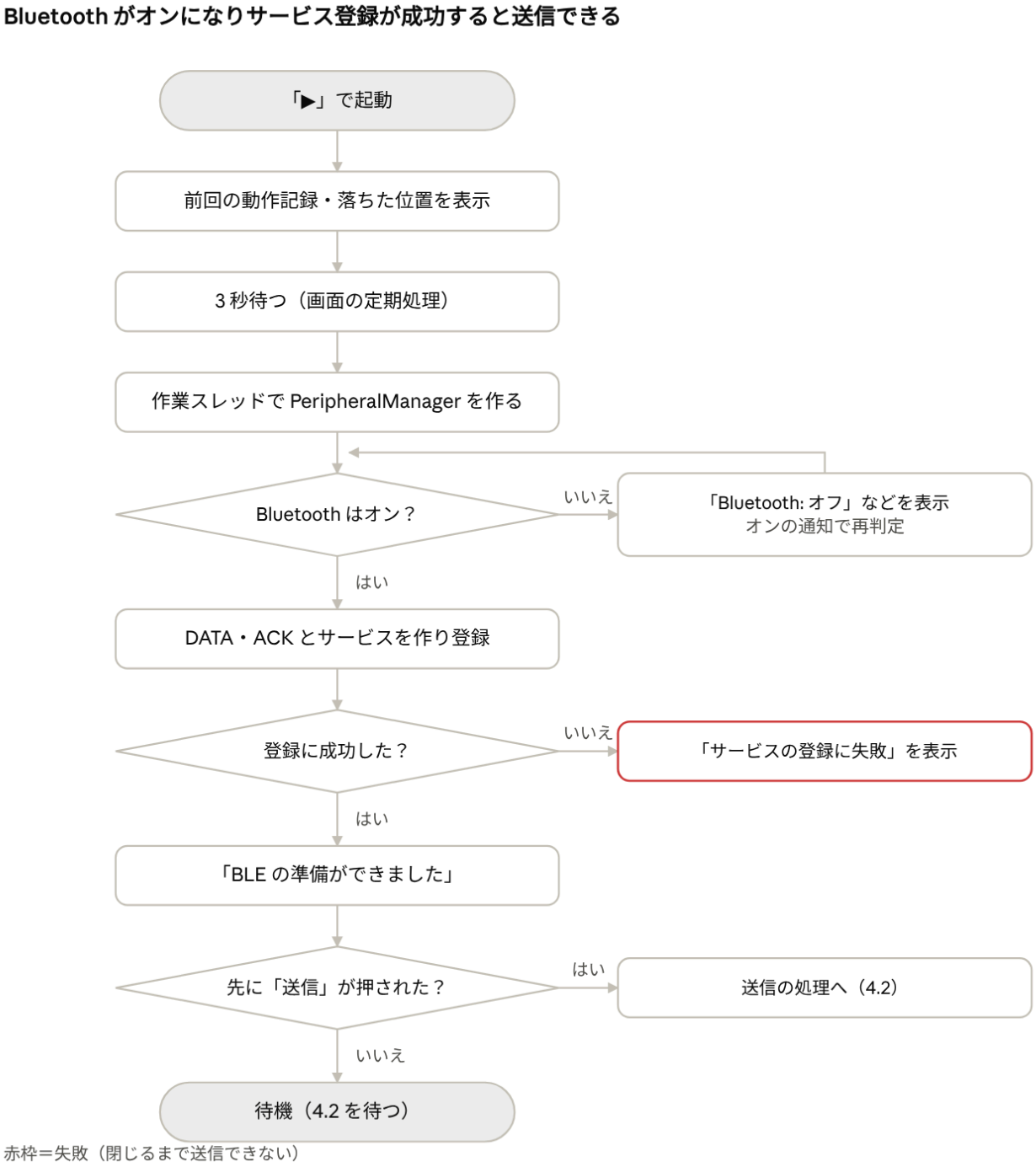
iPhone（受信）の状態遷移 · 6 状態

受信は 1 回ごとに切断して探索中へ戻るため、iPad から続けて何度でも送れます。

4. 状態ごとの処理フロー

3 章の状態をまとめて 4 つの処理ブロックに分け、それぞれの判断と分かれ道を示します。赤枠は失敗またはエラーで、その後どこへ戻るかを矢印で示しています。

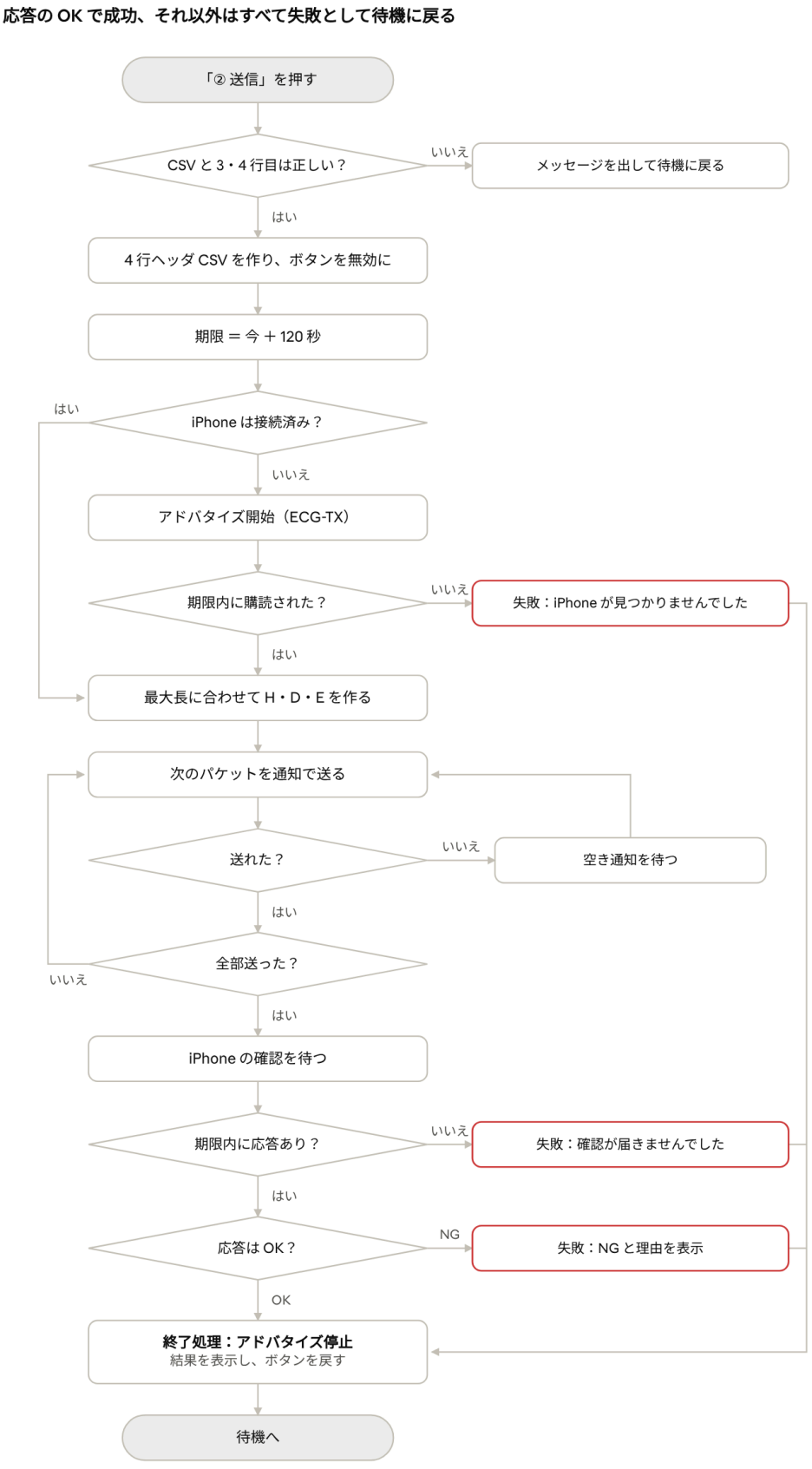
4.1 iPad：起動・BLE 準備（起動 → BLE 準備 → 待機）



iPad：起動・BLE 準備のフロー

Bluetooth がオフや許可なしのときは、オンになった通知で同じ判定をやり直します。

4.2 iPad：送信（待機 → アドバタイズ中 → 送信中 → 応答待ち → 待機）

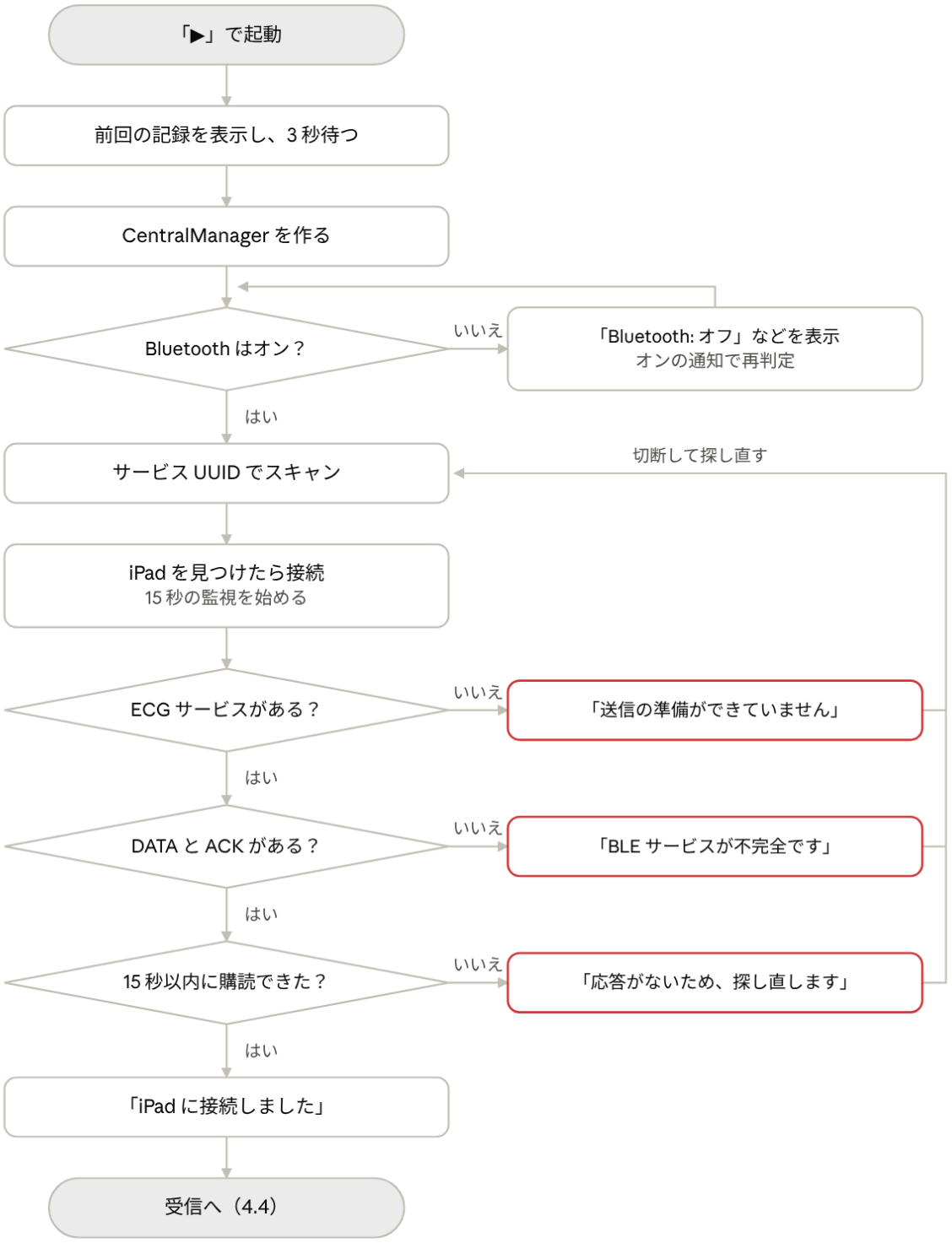


iPad：送信（アドバタイズ中・送信中・応答待ち）のフロー

期限は「送信」を押してから 120 秒で、iPhone の接続待ちと応答待ちの合計に掛かります。

4.3 iPhone：起動・探索・接続（起動 → 探索中 → 接続中）

iPad のサービスと特性がそろったときだけ受信に進む



赤枠＝接続をやめて、もう一度スキャンする。

iPhone：起動・探索・接続のフロー

途中で条件がそろわなければ、その時点で切断してスキャンに戻ります。

4.4 iPhone：受信・保存・応答（受信中 → 保存・応答 → 切断 → 探索中）

E で検証と保存を行い、OK か NG を返してから切断する



受信の途中で接続が切れたときは、途中までのデータを捨てて探索中へ戻る。赤枠＝NG。

iPhone：受信・保存・応答・切断のフロー

H と D は届くたびに確かめ、間違いがあればその場で NG を返します。E が届いたら全体のサイズと CRC32 を確かめて保存します。

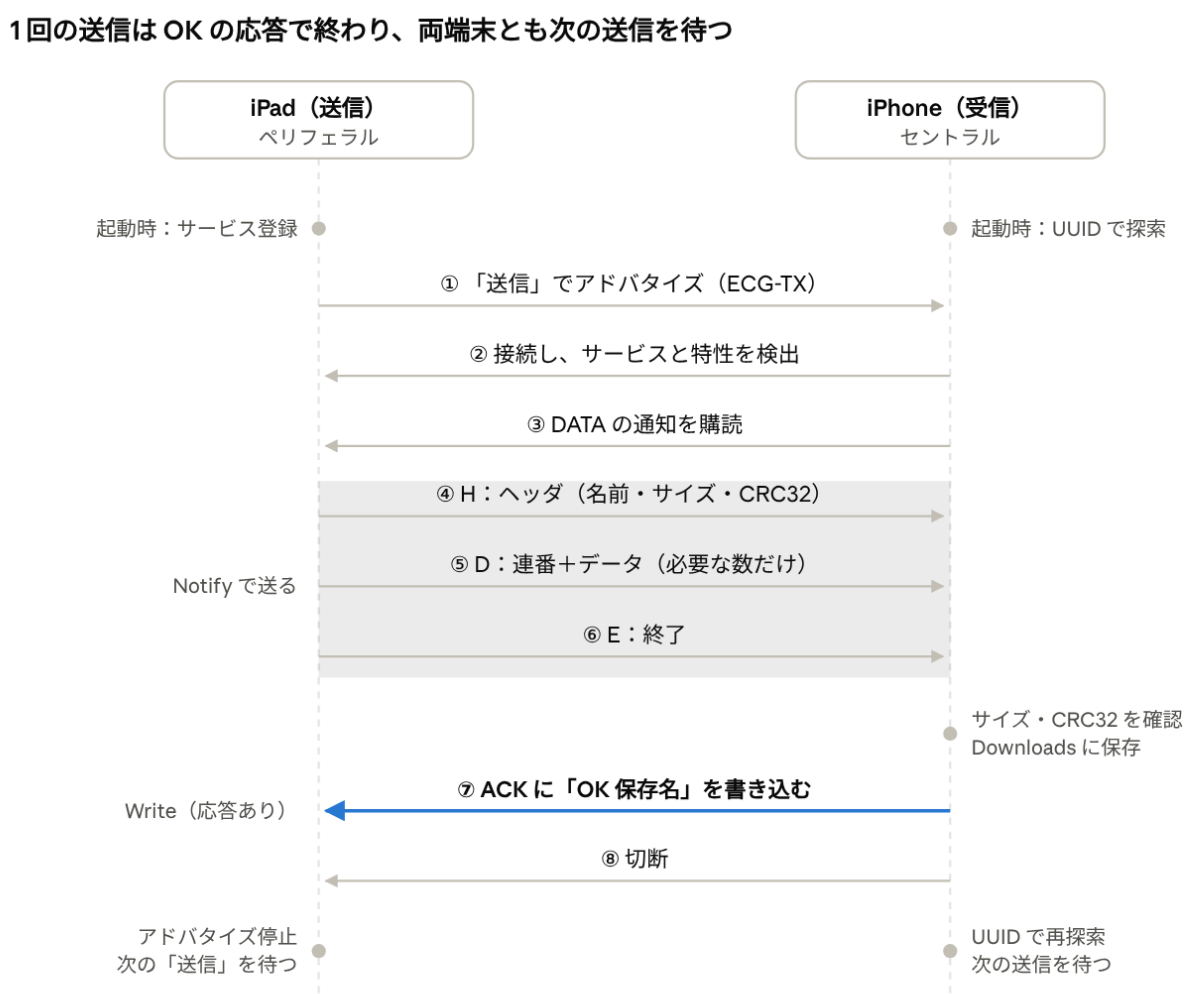
4.5 終了処理（閉じる／×）

端末	手順
iPad	1.「閉じる」を押す 2. 作業スレッドがアドバタイズを止め、サービスを外し、デリゲートを外す 3. PeripheralManager・特性・サービス・iPhone の参照を手放す 4. 作業スレッドの終了を最大 3 秒待つ 5. スクリプトを終える
iPhone	1.「×」を押す 2. スキャンを止め、接続中なら切断する 3. デリゲートを外し、CentralManager・iPad・特性の参照を手放す 4. スクリプトを終える

5. 通信チャート

正常時は①〜⑧の 8 ステップで終わり、異常時も必ず両端末が待機・探索中に戻ります。

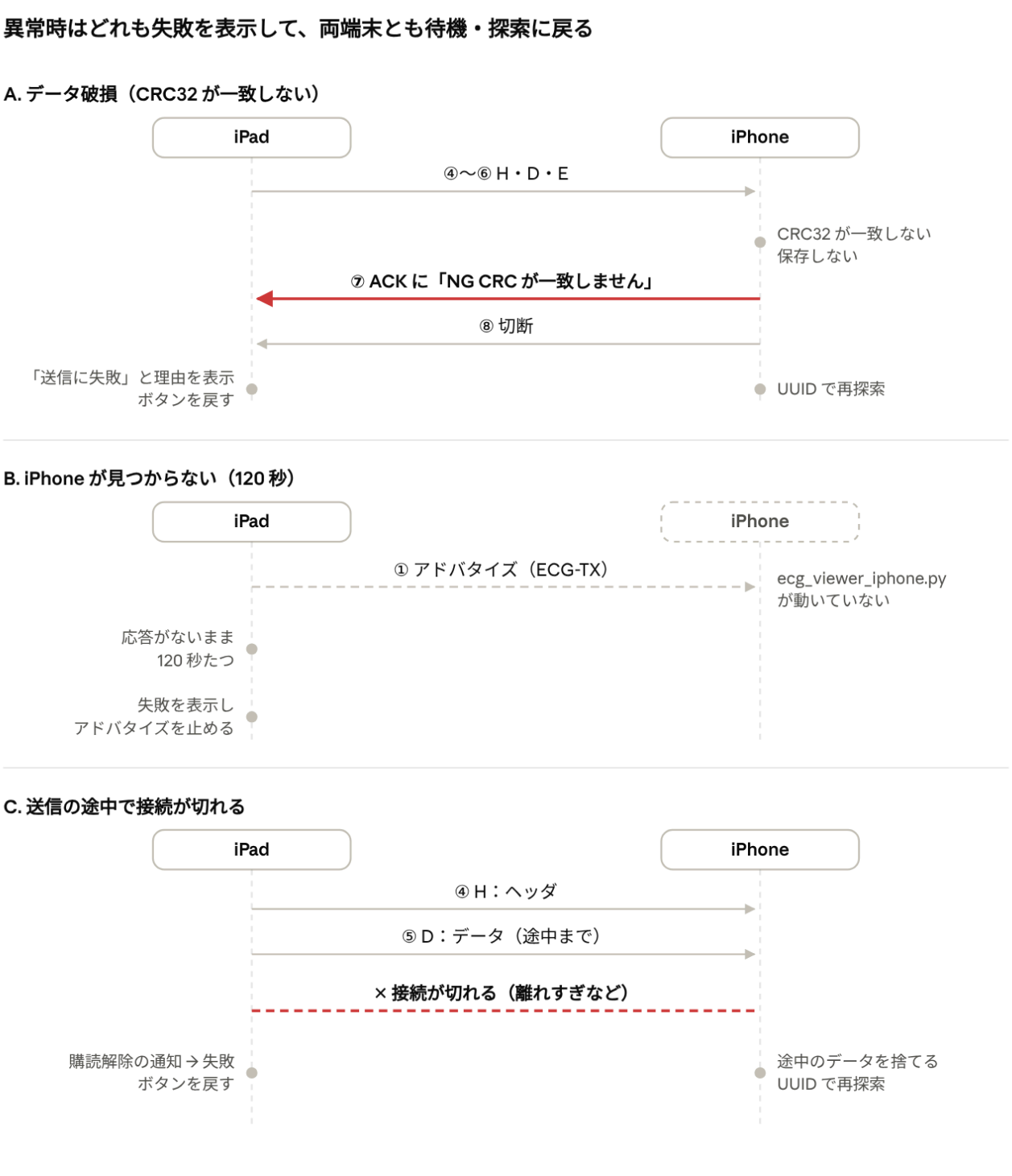
5.1 正常時



送受信の流れ · ①〜⑧

④〜⑥は iPad が待ちなしで続けて送り、⑦の OK を受け取ったら送信成功です。iPhone が先に接続して待っている場合は①〜③を飛ばして④から始めます。

5.2 異常時



異常時の通信 · 3 つの場合

このほか、iPhone が接続して 15 秒以内に特性がそろわないときは、iPhone が切断して探し直します（iPad はアドバタイズを続ける）。

6. 機能一覧と機能の説明

機能は iPad 9 個、iPhone 10 個、共通 3 個の計 22 個です。「状態」は 3 章の状態遷移図のどこで働くかを示します。

6.1 機能一覧

ID	機能	端末	状態	関連する章
F-01	CSV の選択・読み込み	iPad	待機	7.1、8
F-02	ヘッダ判別と波形解析（3・4 行目の計算）	iPad	待機	8、11
F-03	3・4 行目の編集	iPad	待機	7.1
F-04	4 行ヘッダ CSV の作成	iPad	待機 → アドバタイズ中	8
F-05	BLE サービスの公開	iPad	BLE 準備	9.1
F-06	アドバタイズと接続待ち	iPad	アドバタイズ中	9.1
F-07	パケット分割と送信（流量制御）	iPad	送信中	9.2
F-08	受信結果の受け取りと結果表示	iPad	応答待ち → 待機	9.3
F-09	送信の待ち時間の監視（120 秒）	iPad	アドバタイズ中〜応答待ち	10
F-10	iPad の自動探索・接続	iPhone	探索中 → 接続中	9.1
F-11	接続の監視（15 秒）	iPhone	接続中	10
F-12	パケットの受信・組み立て	iPhone	受信中	9.2
F-13	受信データの検証（連番・サイズ・CRC32）	iPhone	受信中 → 保存・応答	9.3
F-14	Downloads への保存	iPhone	保存・応答	9.3
F-15	受信結果の応答（OK／NG）	iPhone	保存・応答	9.3
F-16	切断と再探索（続けて受信）	iPhone	切断 → 探索中	5
F-17	波形表示（縦・横の切り替え）	iPhone	どの状態でも	7.2、7.3
F-18	異常検知の表示（電極・不整脈・心拍）	iPhone	どの状態でも	11
F-19	ファイルを開く・ダミー波形・共有・クリア	iPhone	どの状態でも	7.2
F-20	動作記録と落ちた位置の記録	共通	すべて	12.3
F-21	前回の記録の表示	共通	起動	12.3
F-22	終了処理（閉じる／×）	共通	すべて → 終了	4.5

6.2 機能の説明

F-01 CSV の選択・読み込み　「① CSVを選ぶ」で「ファイル」の選択画面を開きます。選んだファイルを画面の定期処理で読み込み、ファイル情報を表示します。読めないときは「読み込みに失敗」を出します。

F-02 ヘッダ判別と波形解析　ヘッダ形式（4 行／2 行／なし）を判別し、R 波と電極はがれを検出します。3 行目＝電極はがれ区間の数、4 行目＝早期拍・休止の数を計算します（元ファイルに値があればそちらを使う）。

F-03 3・4 行目の編集　画面の 2 つの入力欄で値を書き換えられます（異常時の iPhone 表示を確かめるため）。0 以上の整数でなければ送信しません。

F-04 4 行ヘッダ CSV の作成　「② 送信」で、`ECG`・周波数・3 行目・4 行目・心電データの CSV を作ります。サイズは 1 バイト〜20 MB です。

F-05 BLE サービスの公開　Bluetooth がオンになったら、DATA（Notify）と ACK（Write）を持つサービスを登録します。登録したものは閉じるまで保持します。

F-06 アドバタイズと接続待ち　送信するときだけ `ECG-TX` とサービス UUID を発信し、iPhone が DATA を購読するのを待ちます。iPhone がすでに接続していればすぐ送信します。

F-07 パケット分割と送信　iPhone の最大長に合わせて H・D・E に分け、通知で送ります。送信待ちが満杯なら iOS の空き通知を待って続きを送ります。20% ごとに進み具合を表示します。

F-08 受信結果の受け取りと結果表示　ACK への書き込みに応答し、`OK` なら「送信しました。iPhone 側の保存名」、`NG` なら「送信に失敗」と理由を出します。どちらでもアドバタイズを止め、「② 送信」を戻します。

F-09 送信の待ち時間の監視　「送信」から 120 秒以内に結果が来なければ失敗にします（作業スレッドが 0.5 秒ごとに確認）。

F-10 iPad の自動探索・接続　サービス UUID だけで探し、見つけたら接続してサービス・特性を確かめ、DATA を購読します。端末名は使いません。

F-11 接続の監視　接続を始めて 15 秒以内に特性がどろわなければ切断して探し直します。

F-12 パケットの受信・組み立て　H でファイル名・サイズを受け取り、D を順につなぎます。20% ごとに「受信中…」を表示します。

F-13 受信データの検証　サイズの範囲・連番・合計サイズ・CRC32 を確かめます（9.3）。ひとつでも合わなければ、そこで受信を打ち切って NG を返します。

F-14 Downloads への保存　`~/Documents/Downloads` に安全なファイル名で保存します（同名は上書き）。

F-15 受信結果の応答　`OK <保存名>` または `NG <理由>` を ACK に応答ありで書き込みます（最大 180 バイト）。

F-16 切断と再探索　書き込みが完了したら切断し、「受信完了（N 件目）」を出して再び探し始めます。スクリプトを起動し直さずに続けて受信できます。

F-17 波形表示　受信したらそのファイルを表示します。縦向きで概要、横向きで波形と拍の一覧を出し、向きを変えると自動で切り替わります。

F-18 異常検知の表示　電極状態・不整脈検知・心拍/分、異常の一覧を表示します（決め方は 11.3）。

F-19 ファイルを開く・ダミー波形・共有・クリア　保存済みの CSV を開く、確認用の合成波形（20 秒／電極はがれ入り 30 秒）を表示する、共有シートで渡す、表示を消す、の 4 つです。

F-20 動作記録と落ちた位置の記録　処理ごとに記録ファイルへ 1 行ずつ保存し、iOS の中で落ちたときは Python の位置を別ファイルに残します。

F-21 前回の記録の表示　起動直後に前回の記録の最後 25 行と落ちた位置を画面に出し、その 3 秒後に BLE を始めます。

F-22 終了処理　「閉じる」（iPad）または「×」（iPhone）で、アドバタイズ・探索・接続を止め、保持していたオブジェクトを手放してからスクリプトを終えます。

7. 画面と操作（UI）

操作は iPad の「① CSVを選ぶ」と「② 送信（BLE）」の 2 つだけで、iPhone は起動しておけば自動で受信・表示します。

7.1 iPad 送信画面（全画面）

部品	動き
① CSVを選ぶ（ダウンロード）	「ファイル」の選択画面を開く。選んだ CSV を読み込み、波形を解析する
ファイル情報	ファイル名、ヘッダ形式、秒数・点数、解析結果（電極はがれ区間数・不整脈件数）
3行目 電極状態（0＝正常）	送る CSV の 3 行目。元ファイルの値か解析結果が入る。書き換え可（0 以上の整数）

部品	動き
4行目 不整脈の件数（0＝なし）	送る CSV の 4 行目。同上
② 送信（BLE）	送信を始める。終わるまでボタンは按せない（半透明）
記録欄	進み具合（接続・送信中…%・結果）を下に追加していく
閉じる	BLE を片付けてスクリプトを終える

起動すると前回の動作記録を表示し、約 3 秒後に BLE を始めます。「BLE の準備ができました」が出たら送信できます。

iPad 送信画面は「①で選んで②で送る」の 2 操作



iPad 送信画面（説明 A〜F）

操作手順（iPad）

1. Pythonista で `ecg_sender_ipad.py` を開いて▶を押す（iPhone 側を先に起動しておく）
2. 記録欄に「BLE の準備ができました」が出るまで待つ（約 3 秒）
3. 「① CSVを選ぶ」を押し、送る CSV を選ぶ
4. 必要なら 3 行目・4 行目の値を書き換える

5. 「② 送信（BLE）」を押す
6. 記録欄に「送信しました。iPhone 側の保存名: …」が出たら完了。続けて送るときは 3 から繰り返す
7. 終えるときは「閉じる」を押す

7.2 iPhone ビューア（縦向き）

部品	動き
データ検出の枠	受信前は灰色「データ未検出」、受信後は緑色「データ検出」。ファイル名・秒数・ヘッダ値を表示
電極状態／不整脈検知／心拍/分	3 つの値を表示。異常は赤・橙、正常は緑
ファイル／ダミー／ダミー（電極はがれ）	保存済み CSV を開く、または確認用の合成波形（20 秒／30 秒）を表示
共有／クリア	表示中の CSV を共有シート（AirDrop など）で渡す／表示を消す
横向きの案内	データがあるとき「iPhoneを横向きにしてください」
受信の記録	BLE の進み具合を直近 4 件表示
×	受信を止めて終える

縦向きは受信すると緑の「データ検出」になり、横向きを案内する



iPhone 縦向き画面（説明 A〜G）

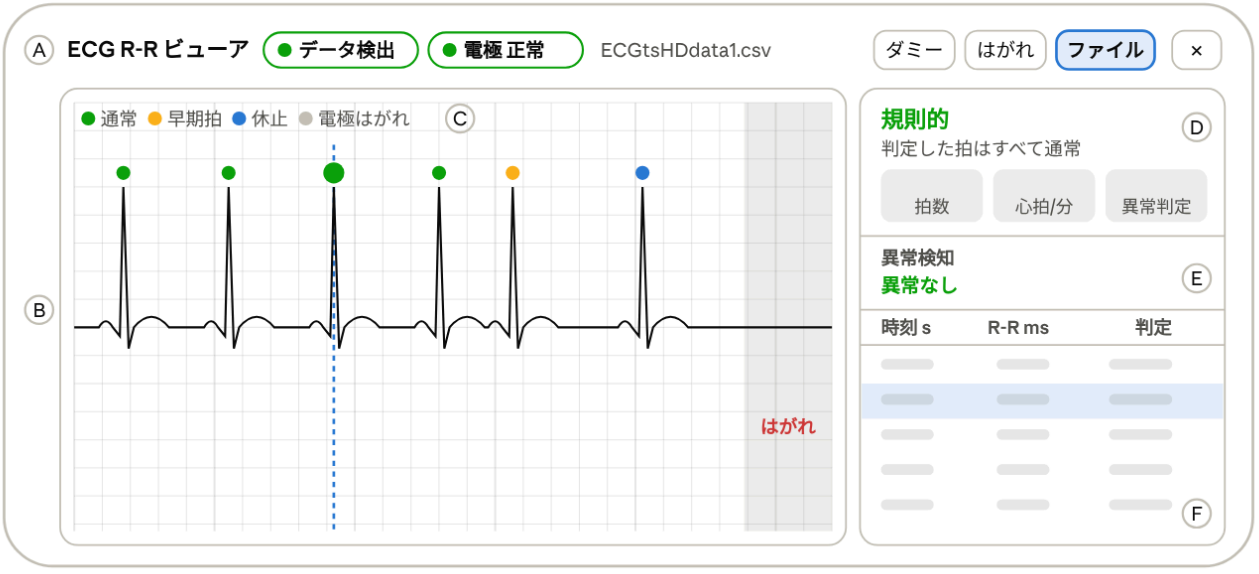
操作手順（iPhone 縦向き）

1. Pythonista で `ecg_viewer_iphone.py` を開いて▶を押す
2. 「BLE 受信待機中」が出たら、画面を前面に出したまま iPad からの送信を待つ
3. 受信すると枠が緑の「データ検出」になり、3 つの値が出る
4. iPhone を横向きにすると波形画面（7.3）になる
5. 保存済みのファイルを見るときは「ファイル」、確認用の波形は「ダミー」を押す
6. 「共有」で AirDrop などに渡す。「クリア」で表示を消す
7. 終えるときは「×」を押す

7.3 iPhone ビューア（横向き）

- 上段：「データ検出」「電極 正常／はがれ」のバッジ、ファイル名・秒数、ダミー／はがれ／ファイルのボタン
- 波形：心電図用紙の格子（細線 0.2 秒・太線 1 秒）、R 波の印と R-R（ms）、電極はがれ区間の網掛け。印の色は通常＝緑・早期拍＝橙・休止＝青・電極はがれ＝灰
- 右のパネル：判定（規則的／不整脈の疑い／判定不可）、拍数・心拍/分・異常判定数、異常検知の一覧（最大 4 件）、拍ごとの時刻・R-R・判定の一覧
- 拍の一覧の行をタップすると、波形上にその拍の位置を点線で示す

横向きは波形と拍ごとの判定を一画面で見せる



Ⓐ 上段：データ検出・電極状態のバッジ、ファイル名、ダミー／はがれ／ファイル、×
Ⓑ 波形：心電図用紙の格子（細線 0.2 秒・太線 1 秒）、R 波の印と R-R（ms）、電極はがれの網掛け
Ⓒ 凡例：印の色（緑＝通常、橙＝早期拍、青＝休止、灰＝電極はがれ）
Ⓓ 判定（規則的／不整脈の疑い／判定不可）と拍数・心拍/分・異常判定数
Ⓔ 異常検知：電極はがれ・不整脈疑いの一覧（最大 4 件）
Ⓕ 拍の一覧：時刻・R-R・判定。行をタップすると波形に点線で位置を示す

iPhone 横向き画面（説明 A〜F、波形はイメージ）

操作手順（iPhone 横向き）

1. データを表示した状態で iPhone を横向きにする

2. 波形の R 波の印の色を見る（緑＝通常、橙＝早期拍、青＝休止、灰＝電極はがれ）
3. 右のパネルで判定と異常検知の一覧を確かめる
4. 拍の一覧の行をタップすると、その拍の位置が波形に点線で出る（一覧が長いときは上下にスクロール）
5. 縦向きに戻すと概要画面（7.2）に戻る

8. CSV データ形式

iPad は必ず 4 行ヘッダ形式にそろえて送ります。元ファイルは 4 行ヘッダ、2 行ヘッダ、ヘッダなしのどれでも読めます。

行	内容	値
1	データ種別	`ECG`（大文字小文字は問わない）
2	サンプル周波数	`256`（Hz）
3	電極状態	0＝正常、0 以外＝異常
4	不整脈	0＝なし、0 以外＝検出した件数
5 ～	心電データ	1 行 1 値（整数）

```
ECG
256
0
0
-12
-8
35
```

- 元ファイルが 2 行ヘッダ（`ECG` ／ `256` の直後から心電データ）なら、iPad が波形を解析して 3・4 行目を作る（3 行目＝電極はがれ区間の数、4 行目＝早期拍・休止の数）
- 自動判別：3・4 行目が 0～9999 の整数なら 4 行ヘッダ、そうでなければ 2 行ヘッダとみなす
- `ECG,256` の 1 行ヘッダ、区切りの `,`・タブ・`;`、先頭の BOM にも対応
- 256 点（1 秒）未満は「サンプルが少なすぎます」で読み込み失敗
- 256 Hz 以外は線形補間で 256 Hz にそろえてから解析する

9. BLE 通信仕様

iPad がサービスを 1 つ公開し、ファイルは DATA の Notify で送り、受信結果は iPhone が ACK へ書き込みます。

9.1 サービスとキャラクタリスティック

名前	UUID	向き	性質／権限	用途
サービス	`8F1D0001-2C5A-4E7B-9A63-5E0C6A1B2D01`	—	プライマリ	iPhone はこの UUID だけで iPad を探す
DATA	`8F1D0002-2C5A-4E7B-9A63-5E0C6A1B2D01`	iPad → iPhone	Notify（0x10）／Readable（0x01）	ファイルデータ（1 通知＝1 パケット）

名前	UUID	向き	性質／権限	用途
ACK	`8F1D0003-2C5A-4E7B-9A63-5E0C6A1B2D01`	iPhone → iPad	Write（0x08）／Writeable（0x02）	受信結果（応答ありの書き込み）

アドバタイズには名前 `ECG-TX` とサービス UUID を載せます。アドバタイズするのは「送信」を押してから送信が終わるまでです。

9.2 パケット形式

1 パケットの最大長は iPhone が受け取れる長さ（`maximumUpdateValueLength`、例 182 バイト）です。

種類	中身	個数
ヘッダ	`H` + JSON `{"name","size","crc","fmt"}`（UTF-8、空白なし）	1
データ	`D` + 連番（4 バイト、ビッグエンディアン、0 から）+ データ（最大長 − 5 バイト）	サイズに応じて
終了	`E`	1

- `size` はバイト数、`crc` は CRC32、`fmt` はヘッダ形式（送信時は `"4"`）
- ヘッダが 1 通知に収まらないときはファイル名を後ろから短くする。最大長が 16 バイト未満なら送信しない
- 送信待ちがいっぱい（`updateValue` が失敗）のときは、iOS の「送信できる」通知を待って続きから送る

9.3 受信側の確認と応答

確認	失敗時の応答
サイズが 1 バイト〜20 MB	`NG サイズが不正です`
ヘッダより前にデータが来ない	`NG ヘッダを受け取る前にデータが届きました`
連番が 0, 1, 2 … と続く	`NG データの順番が不正です`
受け取った量が宣言を超えない・一致する	`NG サイズが一致しません`
CRC32 が一致する	`NG CRC が一致しません（データ破損）`
保存できる	`NG 保存に失敗 …`

すべて通ると `OK <保存名>` を返します。応答は UTF-8 で最大 180 バイトです。保存先は iPhone の `~/Documents/Downloads`（ファイルアプリ：このiPhone内 > Pythonista 3 > Downloads）で、同名ファイルは上書きします。ファイル名から `¥ / : * ? " < > |` を除き、拡張子が `.csv`・`.txt` でなければ `.csv` を付けます。

10. 待ち時間・エラー処理

待ち時間は 2 つで、どちらも失敗したら自動で次の送受信を待つ状態に戻ります。

待ち時間	値	対象	超えたとき
送信の待ち時間（`SEND_TIMEOUT`）	120 秒	iPad：「送信」から iPhone の受信結果が届くまで	送信失敗にしてアドバタイズを止め、ボタンを戻す
接続の待ち時間（`CONNECT_TIMEOUT`）	15 秒	iPhone：接続を始めてから iPad のサービスが見つかるまで	接続を切って探し直す

画面のメッセージ	端末	意味・対処
「iPhone が見つかりませんでした（120 秒）」	iPad	iPhone で `ecg_viewer_iphone.py` が動いていない。起動してもう一度送信
「iPhone から受信の確認が届きませんでした」	iPad	送信後に応答がない。もう一度送信
「送信の途中で iPhone との接続が切れました」	iPad	離れすぎ・画面が切り替わったなど。もう一度送信
「送信に失敗: NG …」	iPad	iPhone が受信を拒否（9.3 の確認）。理由が続けて出る
「Bluetooth: 許可されていません」	両方	設定 → Pythonista → Bluetooth をオン
「Bluetooth: オフ」	両方	設定アプリで Bluetooth をオン
「iPad から応答がないため、探し直します」	iPhone	接続の待ち時間切れ。自動で再試行
「… は送信の準備ができていません」	iPhone	iPad で `ecg_sender_ipad.py` が動いていない
「受信の途中で iPad との接続が切れました」	iPhone	途中までのデータを捨て、次の送信を待つ
「BLE エラー: …」	両方	プログラム内の予期しないエラー。画面とコンソールに内容を出し、動作は続ける

Pythonista が落ちた場合は、同じスクリプトをもう一度起動すると、「前回の動作記録」と「前回落ちた時の Python の位置」が画面に出ます（12.3）。

11. 波形解析

iPad（送る前の 3・4 行目の計算）と iPhone（表示）は同じ解析を使います。整数演算で、`rr_detect.c` と同じ結果になります。

11.1 R 波検出と不整脈判定

1. 2 点前との差の絶対値（上限 65535）を 16 点分足した値を作る
2. 最初の 2 秒（512 点）で、信号の強さ（最大の半分）と雑音の強さ（平均）を決める
3. しきい値 ＝ 雑音 + (信号 − 雑音) ÷ 4。山がしきい値を超え、前の R 波から 51 点（約 200 ms）以上離れていれば R 波
4. 信号・雑音の強さは山ごとに 1/8 ずつ追従させる
5. 直前 8 拍の R-R の中央値と比べて判定する（8 拍たまるまでは「履歴不足」）

判定	条件	表示色
通常	下のどちらでもない	緑
早期拍	R-R ＜ 中央値 × 0.8	橙
休止	R-R ＞ 中央値 × 1.3	青

心拍/分 ＝ 60 × 256 ÷ R-R の平均（点）です。R-R（ms）＝ R-R（点）× 1000 ÷ 256 で表示します。

11.2 電極はがれ検出

波形を 0.5 秒（128 点）ごとに区切って調べます。

種類	条件
平坦	振幅（最大 − 最小）が 30 未満の区切りが 4 つ（2 秒）以上続く
飽和	絶対値が 30000 以上の区切りが 2 つ（1 秒）以上続く

はがれ区間とその前後 0.5 秒の拍、および区間直後の最初の R-R は「保留（電極）」とし、不整脈判定と心拍数から除きます。

11.3 iPhone の表示の決め方

項目	優先順
電極状態	CSV 3 行目が 0 以外 →「異常」。次に解析ではがれあり →「はがれ」（最後 0.6 秒以内にかかるとき）または「はがれ有」。どちらもなければ「正常」
不整脈検知	CSV 4 行目の件数。なければ解析の早期拍・休止の数。8 拍未満なら「判定不可」
判定（横向き）	判定できた拍がない →「判定不可」、早期拍・休止あり →「不整脈の疑い」、それ以外 →「規則的」

12. 内部設計

Pythonista 3.4 で落ちないように、iPad は BLE の操作を専用の作業スレッドにまとめ、CoreBluetooth は型を固定した関数で直接呼びます。

12.1 スレッド構成

スレッド	iPad（送信）	iPhone（受信）
スクリプト	画面を出し、閉じるまで `threading.Event().wait()` で待つ	同左
メイン（画面）	ボタン操作、0.2 秒ごとの `update()` で記録・結果を画面に出す。iOS からの通知は値を取り出して作業スレッドへ渡すだけ	CoreBluetooth の操作と通知の処理をすべてここで行う（予約は `ui.delay`）
BLE 作業（`ECGBleSender`）	CoreBluetooth の操作（サービス登録・アドバタイズ・送信）をすべて行う。0.5 秒ごとに待ち時間も確かめる	なし
ファイル選択	選択画面を出し、選ばれたパスを `update()` に渡す	選択画面を出す

iPad の画面は他のスレッドから `ui.delay` を使いません（予約したスレッドが終わった後に落ちたため）。iPhone のリクエストへの応答（`respondToRequest`）は、要求が有効な通知の中で返します。

12.2 CoreBluetooth の呼び出し方

- Pythonista の `cb` モジュールは使わない（セントラル専用で、書き込みが `PY_SSIZE_T_CLEAN` エラーになる）
- `_send(obj, sel, sig, *args)` が、呼び出しの型ごとに専用の `objc_msgSend` を ctypes で作って呼ぶ。型の記号は `@` ポインタ・`B` BOOL・`q` 整数・`Q` 符号なし整数・`*` C 文字列・`v` なし
- `objc_util` はデリゲートのクラス作成（`create_objc_class`）と `load_framework` だけに使う（共有の `objc_msgSend` の型設定が書き換わり、ArgumentError や強制終了になったため）
- デリゲートの関数はモジュール直下に置き、動作中の送信側／受信側を `_ACTIVE` から探して呼ぶ（入れ子の関数は iPad で NameError）

- オブジェクトの参照は自分で持つ：iPad は登録したサービス・特性・接続中の iPhone を、iPhone は接続中の iPad と特性を、閉じるか切断するまで retain する

12.3 動作記録

ファイル	中身
`ecg_ble_trace_sender.txt` ／ `ecg_ble_trace_receiver.txt`	処理ごとの記録（1 行ごとに保存）。前回分は `_prev.txt`
`ecg_ble_fault_sender.txt` ／ `ecg_ble_fault_receiver.txt`	iOS の中で落ちたときの全スレッドの Python の位置（faulthandler）

保存先はスクリプトと同じフォルダーで、書き込めなければ Pythonista の Documents です。起動時に前回の記録の最後 25 行と落ちた位置を画面に出してから、3 秒後に BLE を始めます。

13. 制限事項とテスト

13.1 制限事項

- 転送中は両方の端末で Pythonista を前面に出したままにする（バックグラウンドでは BLE が止まる）
- 1 回の送信で送れるのは 1 ファイル、1 台の iPhone まで。ファイルは最大 20 MB
- iPhone で同じ名前のファイルを受け取ると上書きする
- 通信はペアリング・暗号化をしていない。近くで同じプログラムを動かしている端末は受け取れるので、個人の心電データを扱う場合は注意する
- 調査用の動作記録を処理ごとに保存しているため、その分だけ処理が遅くなる
- 転送速度は未測定（受信側の最大長と iOS の送信待ちによる）
- 波形解析は確認用の簡易判定で、診断には使わない

13.2 テスト

種類	内容	結果
自動テスト（PC）	`python3 test_ecg_ble.py`：実際の CSV の往復、小さい最大長、データ破損の NG、危険なファイル名、模擬 CoreBluetooth での一連の流れ・終了処理・2 回連続送信・待ち時間切れ、作業スレッドでの実行、動作記録、型ごとの呼び出し	18 件すべて成功
実機診断	`ble_diag_ipad.py`：BLE の準備 12 段と iPhone の接続	すべて OK
実機転送	iPad → iPhone で CSV を送り、波形表示と終了処理を確認	問題なく転送

14. 改訂履歴

Wi-Fi（TCP）版を BLE に置き換え、r17 で実機転送が完成しました。履歴は新しい順です。

版	主な変更
r17	iPad の CoreBluetooth 操作を専用の作業スレッドに移す（iPhone 接続時に落ちる問題を解消）。iPad 画面を `update()` で更新。落ちた位置の記録（faulthandler）

版	主な変更
r16	動作記録を書けないときは Documents に保存。起動診断を追加
r15	動作記録をスクリプトの隣に置き、BLE 開始前に前回分を表示
r14	起動時に前回の動作記録を表示。ObjC の呼び出しをすべて記録
r13	登録したサービス・特性を手放さず保持
r12	CoreBluetooth を型固定の専用 `objc_msgSend` で呼ぶ（ArgumentError 対策）
r11	`objc_util` をメインスレッドだけで使う
r10	落ちた場所を調べる動作記録を追加
r9	iPhone の名前による探索をやめ、サービス UUID だけで探す
r8	送信の待ち時間をメインスレッドで予約
r7	転送後の終了処理を両端末で定義（iPhone は切断して次を待つ、iPad はアドバタイズを止める）
r6	デリゲートをモジュール直下の関数にする（NameError 対策）
初版	Wi-Fi（TCP）の送受信を BLE に置き換え